

Apache Maven | POM, Dependencies & Lifecycle

→ Maven is a tool which helps in managing the java project

classpath → parameter which tells the java compiler & JVM where to look for user-defined, class files, packages & third party libraries (jar files)

POM.xml

→ It provides complete information about the project to maven (like name, version, external dependencies, plugins...)

→ `<project>` `</project>` is the root tag

schema design / rules

→ `xsi:schemaLocation` enforces the schema design & rules the POM will follow.

↳ for example what xml tags are allowed, not allowed

→ `<modelVersion>` `</modelVersion>` ✓

→ `<properties>` `</properties>` ✓

→ `<abc>` `</abc>` ✗ → random self designed tag

→ pom will throw error

→ `<modelVersion>` → what is the version of the POM.xml

→ `groupId`, `artifactId` & `version` tags help us to uniquely identify the project

→ groupId → If you make a JAR file & upload it over the internet, your `groupId` can not conflict with any existing JAR's `groupId`.

This is such that if someone wants to use your library from the internet, they are able to identify independently
↳ belonging to a certain domain/company

`<groupId> in. coderarmy </groupId>`
 └───
 reversed

for example your website is `www. groecbydeepa. com`
 └───
 domain name

⇒ `<groupId> com. groecbydeepa </groupId>`

→ artifactId : Name of the actual project

properties

`<properties> ... </properties>`
↳ contains key-value pairs

`<properties>` ↙ key
 `<mvn. compiler. source> 21 </mvn. compiler. source>`
 ⋮ ↘ value
`</properties>`

→ there are no strict rules regarding what key value pairs you want to add in the properties tag

`<author. name> Keshav Dutt Kaushik </author. name>` ✓

In fact, these properties can be used anywhere in the pom.xml using this syntax -

```
<version> 2.0-SNAPSHOT-${author.name} </version>
```

packaging

→ can be jar or war

```
<packaging> jar </packaging>
```

plugin

→ For maven to execute it's lifecycle commands in the project, it uses plugins

```
<build>
|
|   <plugins>
|   |
|   |   <plugin>
|   |   |
|   |   |   ⋮
|   |   |   </plugin>
|   |   </plugin>
|   </plugins>
</build>
```

parent tag

- pom.xml file supports inheritance.
- Just like everything in Java is an object, which has it's highest parent as Object, pom also has a parent pom (Super pom)
- Each pom.xml inherits from it's parent Super pom, which is the default behaviour that needs no mentioning.

→ For instance, when you create any class in Java and do not mention its parent, it inherits directly from the parent Object class

→ The super pom, or effective pom contains a lot, a LOT of tags which the pom.xml inherits

→ It can override a certain tag, and maven will use the overridden implementation instead.

→ For example, effective pom contains <repository> tag which specifies from which internet source to actually download the external jar from!

Repositories

→ There might be a scenario where you want to fetch your external artifacts from a different remote repository rather than maven central.

→ Your company could have a private server where they want to keep secure JAR's ...

```
<repositories>
|
|   <repository>
|   |   <id> company-repo </id>
|   |   <url> http://repo.codersarmy.in/maven </url>
|   </repository>
</repositories>
```

→ this will now override the parent pom's repository tag

Maven Lifecycle

The java programs that we write goes through a lot of phases that we don't see which maven handles

→ Maven divides the lifecycle into three parts

↳ clean (clean)

↳ default (most important → has multiple phases)

↳ site (site)

default lifecycle

validate → compile → test → package → verify → install → deploy

These phase occur in this order

→ Whatever phase you execute, all the phases present before it will execute first, and consequently the one you want

validate (mvn validate)

→ Very first stage in the default build cycle

→ Verify project is correct

→ All info required to complete build is present

pom validation

→ correct xml schema

→ check syntax / configuratⁿ errors

Dependency Check

→ all declared project dependencies & plugins exist & are available to download

Project Structure

→ Confirms whether the project follows standard directory layout structure (src/main/java/...)

compile (mvn compile)

- Here, maven compiles source code into bytecode (target/classes)
- It does not compile nor execute the unit tests

test (mvn test)

- Here, maven executes unit tests against compiled source code

package (mvn package)

- Maven takes compiled source code & bundles into its distributable format (JAR, WAR, EAR) inside target folder

verify (mvn verify)

- Designed to run checks on the results of integration tests
- Ensure specific code quality criteria are met (whether your code is using extra space, unused variables etc.)
- By default, the verify phase does not contain pre-configured core goals*, making it an intentional hook for developers to bind strict quality analysis plugins

install (mvn install, mvn clean install)

- Here, maven takes the packaged artifact (created during package phase) and installs it locally in your computer's repository folder

deploy (mvn deploy)

- Final stage in the default build lifecycle
- Maven copies the final packaged artifact to a remote repository (where your companies other artifacts might be present) such that it can be shared with other developers & projects.

Under the Hood (Phase vs Goal)*

- A phase in maven is just a conceptual step
- The actual heavy lifting is done by plugin goals
- Depending upon the packaging type defined in the pom.xml, maven binds different goals to a phase by default.

Clean lifecycle (mvn clean)

- Maven cleans the workspace by removing all files & artifacts generated from previous builds by removing the target folder
- note that, the packaged artifacts installed in the local cache (C.m2 / repo ...) are not removed. Only the ones present in current workspace

Site lifecycle

- In this lifecycle, maven generates project documentation, reports and websites

